

TP : Fonctionnalités avancées d'ORM

Contexte du projet

Nous développons une **application simple de gestion de tâches (Task Manager)**. Les tâches contiennent : titre, description, statut (pending / in_progress / completed par défaut "pending"), priorité (défaut 1), date d'échéance, association à un utilisateur (User).

1 : Création du projet et configuration

```
composer create-project laravel/laravel 016-gestion-taches
cd gestion-taches
```

Lancer le serveur :

```
php artisan serve
```

2 : Création du modèle et de la migration

```
php artisan make:model Task -m -f
```

Éditer la migration database/migrations/xxxx_create_tasks_table.php :

```
public function up()
{
    Schema::create('tasks', function (Blueprint $table) {
        $table->id();
        $table->string('title');
        $table->text('description')->nullable();
        $table->string('status')->default('pending');
        $table->unsignedTinyInteger('priority')->default(1);
        $table->date('due_date')->nullable();
        $table->foreignId('user_id')->constrained()->onDelete('cascade');
        $table->softDeletes();
        $table->timestamps();
    });
}
```

```
php artisan migrate
```

3 : Configuration du modèle Task

app/Models/Task.php :

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;
use Illuminate\Database\Eloquent\Prunable; // Pour pruning

class Task extends Model
{
    use HasFactory, SoftDeletes, Prunable;
```

```

        protected $fillable = ['title', 'description', 'status', 'priority',
        'due_date', 'user_id'];
        // Alternative : protected $guarded = ['id']; // interdit la modification
        massive de certains champs

        protected $attributes = [
            'status' => 'pending',
            'priority' => 1,
        ];

        protected $casts = [
            'due_date' => 'date',
            'priority' => 'integer',
        ];

        public function user()
        {
            return $this->belongsTo(User::class);
        }

        // Méthode pour pruning (tâches complétées depuis plus de 30 jours)
        public function prunable()
        {
            return static::where('status', 'completed')
                ->where('updated_at', '<', now()->subDays(30));
        }

        // Callback optionnel avant pruning (ex: supprimer fichiers joints si besoin)
        protected function pruning()
        {
            // Log ou actions supplémentaires
        }
    }
}

```

4 : Données initiales (Seeder)

```
php artisan make:seeder TaskSeeder
```

database/seeder/TaskSeeder.php (simplifié, utilise un User existant ou en crée un) :

```

public function run()
{
    $user = \App\Models\User::first() ?? \App\Models\User::factory()->create();

    Task::create([
        'title' => 'Rédiger rapport',
        'description' => 'Rapport mensuel',
        'status' => 'pending',
        'priority' => 2,
        'due_date' => now()->addDays(7),
        'user_id' => $user->id,
    ]);

    // Ajouter 4-5 tâches supplémentaires (pending, in progress, completed)
    // Exemple : une complétée depuis longtemps pour tester pruning plus tard
}

```

```
php artisan db:seed --class=TaskSeeder
```

Vérifiez les données : `php artisan tinker → Task::all();`

5 : Démonstration des fonctionnalités (recommandé dans Tinker ou contrôleur)

Valeurs par défaut : Lors de `Task::create(['title' => 'Test'])`, `status = "pending"`, `priority = 1` automatiquement.

refresh / fresh :

```
$task = Task::find(1);
$task->status = 'completed';
$task->save();
$task->refresh(); // Recharge l'instance actuelle depuis la DB
// Ou $freshTask = $task->fresh(); // Nouvelle instance fraîche
```

Pagination (découpage) :

```
$tasks = Task::paginate(10); // ou simplePaginate(), cursorPaginate()
```

Tri (sorting) :

```
$tasks = Task::orderBy('priority', 'desc')
->orderBy('due_date', 'asc')
->get();
```

Examen des changements :

```
$task = Task::find(1);
$task->status = 'in_progress';
$task->isDirty(); // true
$task->isDirty('status'); // true
$task->getDirty(); // tableau des champs modifiés
$task->save();
$task->getChanges(); // changements appliqués
$task->wasChanged('status'); // true après save
$task->wasRecentlyCreated; // true si nouvellement créé
```

Upsert (insert ou update en masse, atomique) :

```
Task::upsert([
    ['id' => 1, 'title' => 'Tâche mise à jour', 'status' => 'completed',
    'user_id' => 1],
    ['id' => 10, 'title' => 'Nouvelle tâche', 'status' => 'pending', 'user_id' =>
    1],
], ['id'], ['title', 'status']); // unique sur id, update title + status
```

Truncate (supprime tout, très rapide, attention !) :

```
Task::truncate();
```

Suppression en masse :

```
Task::whereIn('id', [1, 2, 3])>delete(); // soft delete si trait activé
// Ou Task::destroy([1,2,3]);
```

Soft deletes :

```
$task = Task::find(5);
$task->delete(); // soft (deleted_at rempli)
$task->forceDelete(); // suppression définitive
$task->restore(); // restauration
```

```
Task::onlyTrashed()->get();           // uniquement supprimées
Task::withTrashed()->get();           // tout
```

Pruning (suppression définitive des anciens enregistrements) : Le trait est déjà dans le modèle. Exécuter manuellement :

```
php artisan model:prune
```

Réplication (duplicate d'une tâche) :

```
$task = Task::find(1);
$copy = $task->replicate();           // copie attributs (sauf id, timestamps,
soft delete fields)
$copy->title = 'Copie de ' . $task->title;
$copy->save();
```

Comparaison de modèles :

```
$task1 = Task::find(1);
$task2 = Task::find(1);
$task1->is($task2);                  // true (même classe + même clé primaire)
$task1->isDirty();                   // comparaison d'état
```

6 : Events avec closures (dans le modèle)

Dans **Task.php** → **boot()** :

```
protected static function booted()
{
    static::saving(function ($task) {
        if (empty($task->user_id)) {
            $task->user_id = 1;
        }
        Log::info("Tâche {$task->id} en cours de sauvegarde avec le statut
'{$task->status}'", ['title' => $task->title]);
    });

    static::updated(function ($task) {
        if ($task->wasChanged('status') && $task->status === 'completed') {
            // Ex : envoyer notification, log, etc.
            Log::info("Tâche {$task->id} complétée");
        }
    });
}
```

7 : Events avec Observer

```
php artisan make:observer TaskObserver --model=Task
```

app/Observers/TaskObserver.php (exemple) :

```
public function created(Task $task)
{
    // Actions après création
}

public function updated(Task $task)
{
    // ...
}
```

```
public function deleted(Task $task)
{
    // ...
}
```

Enregistrer dans **AppServiceProvider.php** → boot() :

```
Task::observe(TaskObserver::class);
```

Conseils finaux et tests

- Testez tout dans php artisan tinker ou créez un contrôleur/route simple pour lister/trier/paginer les tâches.
- Pour relations : Task::with('user')->get();
- Attention : truncate/suppression en masse est destructif → utilisez en développement.
- Vérifiez les logs et la DB après chaque test.